SENSAAS and SENSAAS-Flex

Alignment of molecules using colored shape and conformational flexibility

SENSAAS-Bioisostere

3D shape-guided bioisosteric replacements and scaffold-hopping

References:

Biyuzan H., Masrour M.-A., Grandmougin L., Payan F. and Douguet D., SENSAAS-Flex: a joint optimization approach for aligning 3D shapes and exploring the molecular conformation space, *Bioinformatics*, 40, 3, **2024**. Doi: 10.1093/bioinformatics/btae105

Douguet D. and Payan F., SenSaaS: Shape-based Alignment by Registration of Colored Point-based Surfaces, *Molecular Informatics*, **2020**, 8, 2000081. Doi: 10.1002/minf.202000081

License

Copyright

Introduction:

3D shape is an important molecular feature in biological activity. **SENSAAS** (SENsitive Surface As A Shape) compares molecules by aligning point-based representation of the van der Waals surface and provides superimposition of 3D graphs and similarity scores. Further, SENSAAS-Flex perform the flexible alignment of two molecular shapes by optimizing the 3D conformation of the aligned molecule.

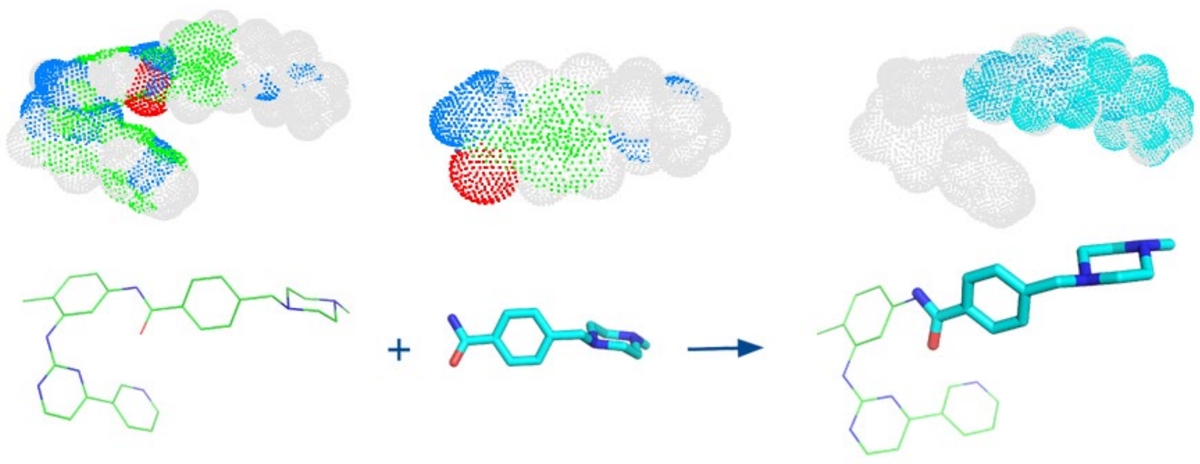
The documentation and the code of the first version of SENSAAS can be found on GitHub at https://github.com/SENSAAS/sensaas-py/blob/main/docs/index.rst. This version allows rigid alignment only.

The documentation and the code for the flexible version of SENSAAS are available at https://github.com/SENSAAS/sensaas-flex.

The documentation and the code for the SENSAAS-Bioisostere are available at https://github.com/SENSAAS/sensaas-bioisostere.

The documentation and tutorials of SENSAAS, SENSAAS-Flex and SENSAAS-Bioisostere are provided below.

What is SENSAAS?



SENSAAS is the result of a collaboration between researchers of two labs of UniCA (University Côte d'Azur): I3S and IPMC. Based on the publication SenSaaS: Shape-based Alignment by Registration of Colored Point-based Surfaces, **SENSAAS** is a shape-based alignment software which allows to superimpose molecules in 3D space.

Main features:

- Uses a surface-based representation that also displays pharmacophore features

- Allows flexible alignments i.e. an optimisation of the conformation of the query molecule

- Allows matchings and submatchings e.g. aligning a fragment on a large molecule

- Provides the superimposition of 3D molecular graphs

- Provides similarity scores

- Is free and open source

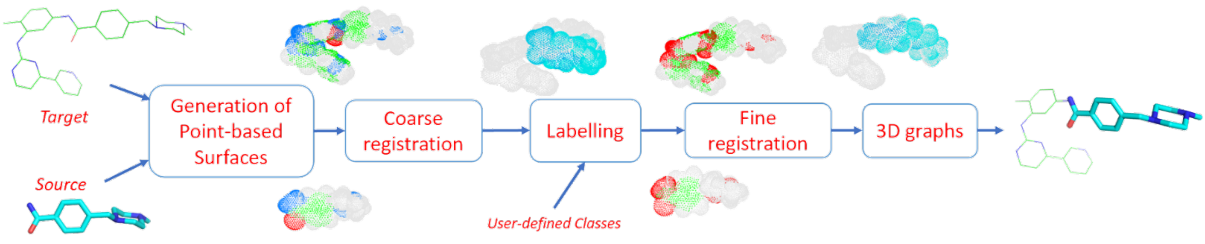
How does SENSAAS work?

3D point clouds or 3D meshes are data structures used in many fields well known by large audience (robotics, 3D reconstruction, games, autonomous navigation...) to model surfaces or volumes. Such 3D representations can also be relevant in chemistry to describe molecules although they were not the most used so far.

SENSAAS (SENsitive Surface As A Shape) is a shape-based alignment software using 3D point-based representation of the van der Waals surface. SENSAAS is an original tool that combines recent methods dedicated to 3D registration, initially developed for the fusion of 3D point clouds, collected by devices such as depth cameras or LiDAR scanners.

Considering two molecules named Source and Target as inputs, SENSAAS gives a transformation matrix as output, leading to the "best" 3D alignment of Source on Target. SENSAAS follows four major steps:

- generation of a point cloud from the molecular surface of each input molecule
- coarse alignment of the two point clouds thanks to a geometry-aware global registration
- labelling of each point of the two clouds according to user-defined classes
- refinement of this alignment by applying a color and geometry-aware local registration. At this step, a color is assigned to each point, in function of its label



1. Generation of input point clouds For each input file (Source and Target), a point cloud of the van der Waals surface is obtained. Each point is described by its 3D coordinates, and a color (RGB) according to the nature of the underlying atom.

2. Coarse alignment by global registration the Source point cloud is globally superimposed on the Target, by finding an initial matching in terms of geometry only. The matching is done by using local 3D descriptors computed on a limited number of points (also called downsampled point clouds). We use the descriptors FPFH (presented in Fast Point Feature Histograms (FPFH) for 3D Registration), and the matching is done with RANSAC (Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography).

3. Labelling of the points of each point cloud Each point is colored according to its belonging to a user-defined class. In the current version, the classes depend on the pharmacophore features, but they could depend on any physico-chemical property mapped onto the surface, or any configuration given by the user.

4. Refinement of the alignment At this step, the registration takes into account the geometry of the point clouds, but also the color of the points assigned by labelling. The coarse alignment is improved by finding the best matching between the two colored point clouds. The method used here is the method of Colored Point Cloud Registration Revisited. This step results into a final transformation matrix (rotation + translation), that is applied to the Source molecule to get the final alignment.

Installing

SENSAAS relies on the open-source library Open3D. The current release of SENSAAS uses **Open3D version 0.12.0** along with **Python3.7**.

Visit the following URL for using Python packages distributed:

- via PyPI: http://www.open3d.org/docs/release/getting_started.html
- or conda: https://anaconda.org/open3d-admin/open3d/files. For example, for windows-64, you can download *win-64/open3d-0.12.0-py37_0.tar.bz2*

Virtual environment for python with conda (for Windows for example)

Install conda or Miniconda.

Launch Anaconda Prompt, then complete the installation:

```
conda update conda
conda create -n sensaas
conda activate sensaas
(sensaas) > conda install python=3.7 numpy
```

Once Open3D downloaded or extracted from the archive:

```
(sensaas) > conda install open3d-0.12.0-py37_0.tar.bz2
```

Some scripts written in Perl are used in SENSAAS-Flex.

```
(sensaas) > conda install perl
```

As well, the open-source Cheminformatics Software RDKit must be installed (eg: version 2022.9.5).

```
(sensaas) > pip install rdkit-pypi
```

Then, retrieve and unzip sensaas-bioisostere repository in your desired folder. The directory containing executables is called sensaas-bioisostere-main.

See below for running the programs **sensaas.py, meta-sensaas.py**, **sensaasflex.py, meta-sensaasflex.py and sensaas-bioisostere.pl**.

Linux

Scripts written in Perl are used in SENSAAS-Flex and SENSAAS-Bioisostere, thus, check for a Perl package on your system.

Install (more information at http://www.open3d.org/docs/release/getting_started.html):

```
1. Python 3.7 and numpy
2. Open3D version 0.12.0
```

The open-Source Cheminformatics Software RDKit must be installed (eg: version v2022.9.5). More information on RDKit can be found at https://www.rdkit.org/docs/Install.html

```
3. RDKit
```

Retrieve and unzip sensaas-bioisostere repository in your desired folder. The directory containing executables is called sensaas-bioisostere-main.

See below for running the programs **sensaas.py, meta-sensaas.py**, **sensaasflex.py, meta-sensaasflex.py and sensaas-bioisostere.pl**.

MacOS

Not tested

Molecular viewer

We suggest you install a molecular viewer so you can visualize the molecular alignments.

PyMOL is a well-known software program for molecular visualisation available for Windows, Linux or macOS. More information at PyMOL Wiki https://pymolwiki.org or https://pymol.org/

Of note, we had trouble installing PyMOL in the current conda environment (python 3.7 and open3D 0.12.0). The installation method described in https://github.com/SENSAAS/sensaas-py no longer works.

List of I/O Formats

In our implementation, input molecules are **3D structures with explicit hydrogen atoms**. Molecules are represented either by their 3D graphs or by their resulting 3D point clouds.

SENSAAS's scripts can read several input file formats:

Input type	**File format**	
sdf	SDF format file	**3D graph**
pdb	PDB format file	(**3D graph**) reads ATOM and HETATM coordinates
dot	PDB format file	(**Point cloud**) reads HETATM lines that contain coordinates of dots and the atom type for defining the label
xyzrgb	xyzrgb format file	(**Point cloud**) ascii file used in 3D data processing such as Open3D; contains coordinates of dots and color
pcd	PCD format file	(**Point cloud**) used in 3D data processing such as Open3D

The output file format depends on the input file format:

- if the Source input file is **sdf** then **Source_tran.sdf** is the transformed sdf source file
- if the Source input file is **pdb** then **Source_tran.pdb** is the transformed pdb source file
- if the Source input file is **dot** then **Source-dots_tran.pdb** is the transformed dot file in pdb format
- if the Source input file is **xyzrgb** then **Source_tran.xyzrgb** is the transformed xyzrgb file
- if the Source input file is **pcd** then **Source_tran.pcd** is the transformed pcd file

Colors

In our implementation, labels aim to recapitulate typical pharmacophore features such as aromatic (colored in green), lipophilic (colored in white/grey) and polar groups (colored in red):

- **class 1** (or label 1) includes non polar hydrogen (H) and halogen atoms excepting fluorines (Cl, Br and I). Hydrogen and halogen atoms are molecule endings. They are the most frequent atoms that contribute to the surface geometry and coloration, and thus, highlight the apolar surface area. Points belonging to this class are colored in white/grey.
- **class 2** (or label 2) includes polar atoms able to be involved in hydrogen bonds such as N, O, S, H (if linked to N or O) and F. Points belonging to this class are colored in red.
- **class 3** (or label 3) includes “skeleton elements” such as C, P and B. Points belonging to this class are colored in green.
- **class 4** (or label 4) includes all elements not listed in the first three classes. This class is empty for most small organic molecules in medicinal chemistry. Points belonging to this class are colored in blue.

Fitness scores

The alignments provided by SENSAAS are evaluated by fitness scores calculated from point clouds. A fitness score indicates how many points are paired. Points are considered paired if their distance is lower than a given threshold. In our implementation, we set the threshold value to 0.3 because it is the average distance between two adjacent points in our original point clouds.

Each score is similar to a Tversky coefficient tuned to evaluate the embedding of a point cloud in another one. Therefore, the score of the Source and the score of the Target may differ. The smallest point cloud of the two will always obtain the highest fitness score as more points are paired, proportionally.

There are three different fitness scores, but we only use 2 of them, gfit and hfit, to finally calculate gfit+hfit.

- **gfit** estimates the geometric matching of point-based surfaces. It is the ratio between the number of points of the transformed Source that match points of the Target, and its total number of points - **it ranges between 0 and 1**
- **hfit** estimates the matching of colored points representing pharmacophore features. It is the sum of the fitness for each class except the first class, to specifically evaluate the matching of polar and aromatic points (classes 2, 3 and 4) - **it ranges between 0 and 1**

- cfit is the sum of the fitness for each class, to specifically evaluate the matching of the colored points of the 4 classes - it ranges between 0 and 1

The hybrid score called **gfit+hfit** is the sum = gfit + hfit scores - **gfit+hfit ranges between 0 and 2**

- A gfit+hfit score close to 2.0 means a perfect superimposition.
- A gfit+hfit score close to 0.0 means that there are no similarities between molecular shapes.

Tutorials

We recommend creating a separate directory for your project to avoid mixing executables in sensaas-bioisostere-main with your files.

SENSAAS and SENSAAS-Flex

This tutorial presents several basic usages of SENSAAS with SDF molecular files. In the folder examples, you will find some molecular files as test cases to work with.

P04035-7.sdf is the experimental conformer of the ligand 4HI (statin inhibitor) observed in the protein-ligand complex PDB 3CCW (3-hydroxy-3-methylglutaryl coenzyme A reductase (HMGR); hmg-coa reductase). This example was extracted from the AstraZeneca data set.

P04035-7-confs.sdf contains 10 conformers generated with RDKit.

P04035-7-confs1.sdf is a conformer generated with RDKit.

phenyl.sdf contains a fragment that will be tested in the clustering mode.

I - Run a rigid alignment with sensaas.py

This script allows to align one Source molecule on one Target molecule:

```
sensaas.py <target-type> <target-file-name> <source-type> <source-file-name>
<log-file-name> <mode>
```

<target-type>

type of the Target file (sdf/pdb/dot/xyzrgb/pcd)

<target-file-name>

name of the Target file

<source-type>

type of the Source file (sdf/pdb/dot/xyzrgb/pcd)

<source-file-name>

name of the Source file

<log-file-name>

name of the log file that details the alignment with **scores of Source**.

<mode>

- **optim** executes the alignment and generates a transformation matrix
- **eval** evaluates the superimposition "in place" (without aligning)

a) Example with the 'optim' mode

The following example works with two molecules from the directory examples/

```
sensaas.py sdf examples/P04035-7.sdf sdf examples/P04035-7-confs1.sdf
slog.txt optim
```

You may have to run the script as follows:

```
python sensaas.py sdf examples/P04035-7.sdf sdf examples/P04035-7-confs1.sdf
slog.txt optim
```

Note:
Don't worry if you get the following warning from Open3D: "*Open3D WARNING KDTreeFlann::SetRawData Failed due to no data.*". It is observed with conda on windows.

Here, the source file P04035-7-confs1.sdf is aligned (**moved**) on the target file P04035-7.sdf (experimental conformation) (**that does not move**).

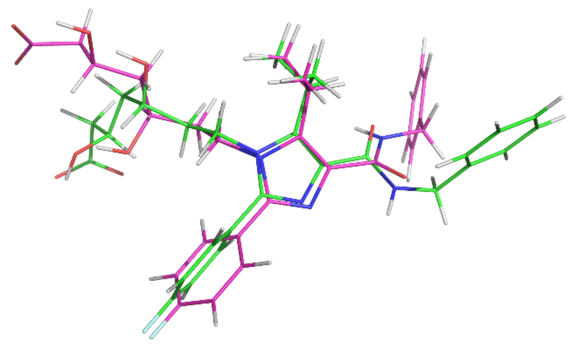
- The output file **Source_tran.sdf** contains the aligned (transformed) coordinates of the Source.
- The output file **tran.txt** contains the transformation matrix applied to the input Source file.
- The **slog.txt** file details results with final scores of the aligned Source molecule on the last line. In the current example, the last line may look like:

  "`gfit= 0.356 cfit= 0.302 hfit= 0.252 gfit+hfit= 0.608`"

It indicates that the best alignment of the Source file (P04035-7-confs1.sdf) has a fitness score gfit+hfit = 0.61.

Visualization To visualize the result, you can use any molecular viewer. For instance, you can use PyMOL if installed to load the Target and the aligned Source:

```
pymol P04035-7.sdf Source_tran.sdf
```



The Source_tran.sdf (in magenta) is aligned on the experimental conformation P04035-7.sdf (in green). The RMSD value between conformers equals 3.36 Å (see RMSD calculation below).

RMSD calculation If two molecules are exactly the same then they possess the same 3D graph. In such case, the root-mean-square distance (RMSD) of corresponding atom pairs in 3D graphs can be calculated using two RDKit based tools (both are present in the folder utils).

The first one is SymmFit (author: Paolo Tosco; the code can be found at https://www.mail-archive.com/rdkit-discuss@lists.sourceforge.net/msg04915.html) which allows the minimization of RMSD value but only when the atoms in the two structure files are arranged in the same order.

Syntax (‘in place’ RMSD calculation):

```
rdkit-symmFitRMSD.py -r mol1.sdf mol2.sdf
```

Syntax (minimizes RMSD and writes transformation matrix called rdkit-tran.txt):

```
rdkit-symmFitRMSD.py -s mol1.sdf mol2.sdf
```

The second one is CalcLigRMSD (author: Carmen Esposito; the code can be found at https://github.com/cespos/rdkit/tree/add-CalcLigRMSD-for-prealigned-compounds/Contrib/CalcLigRMSD) which allows the calculation of RMSD value of ‘in place’ structures (without minimization) whatever the arrangement of atoms in the two structure files.

In the present case study, we use that one because our atoms are not in the same order in the sdf files. We obtain:

```
python rdkit-CalcLigRMSD.py examples/P04035-7.sdf Source_tran.sdf
```

```
RMSD= 3.36
```

b) Example with the 'eval' mode:

Given two molecules, molecule1.sdf and molecule2.sdf, the eval mode evaluates the superimposition "in place" (without aligning)

```
sensaas.py sdf molecule1.sdf sdf molecule2.sdf slog.txt eval
```

Here, the resulting slog.txt contains final scores of molecule2.sdf on the last line.

```
sensaas.py sdf molecule2.sdf sdf molecule1.sdf slog.txt eval
```

Here, the resulting slog.txt contains final scores of molecule1.sdf on the last line.

In our present case study, we can recalculate fitness scores of the aligned Source_tran.sdf (see slog.txt):

```
sensaas.py sdf examples/P04035-7.sdf sdf Source_tran.sdf slog.txt eval
```

You may have to run the script as follows:

```
python sensaas.py sdf examples/P04035-7.sdf sdf Source_tran.sdf slog.txt eval
```

or calculate the score of the Target P04035-7.sdf (see slog.txt):

```
sensaas.py sdf Source_tran.sdf sdf examples/P04035-7.sdf slog.txt eval
```

II - Run rigid alignment(s) with meta-sensaas.py

This script only works with **SDF format files.** It allows to align several Source and Target molecules.

The syntax is:

```
meta-sensaas.py molecules-target.sdf molecules-source.sdf -r nb -s score_type
-l x
```

<molecules-target.sdf>

name of the Target file

<molecules-source.sdf>

name of the Source file

<-s score_type>

Choose the score type {source target mean}:

-s source
here the score of the aligned source will be used to rank solutions and fill matrix-sensaas.txt. This is the default setting if the option -s is not indicated.

-s target
here the score of the target will be used to rank solutions and fill matrix-sensaas.txt.

-s mean
here the mean of the score of the target and of the aligned source will be used to rank solutions and to fill matrix-sensaas.txt. This option is interesting to favor source molecules that have the same size of the Target.

<-l x>

This option sets the database/virtual screening mode. It allows to repeat (loop) x times the alignment using SENSAAS. x is an integer. As SENSAAS is based on stochastic algorithms, one may want to repeat, for example, 2 times a calculation. Results show that it improves alignments when molecules have few similarities. Only the best result is kept.

<-r nb>

Used to execute the clustering mode. nb is an integer. It indicates the number of repeats of SENSAAS before clustering the alignments.

a) Database/Virtual Screening mode

This script is suited to run virtual screenings of sdf files containing several molecules (database mode). For example, if you want to screen a sdf file containing several conformers for Target and/or Source. Solutions can then be ranked in descending order of score and a similarity matrix is provided. The syntax is:

```
meta-sensaas.py molecules-target.sdf molecules-source.sdf
```

Example

```
meta-sensaas.py examples/P04035-7.sdf examples/P04035-7-confs.sdf
```

You may have to run the script as follows:

```
python meta-sensaas.py examples/P04035-7.sdf examples/P04035-7-confs.sdf
```

Note:
Don't worry if you get the following warning from Open3D: "*Open3D WARNING KDTreeFlann::SetRawData Failed due to no data.*". It is observed with conda on windows.

Outputs are:

- the file **bestsensaas.sdf** that contains the best ranked aligned Source
- the file **catsensaas.sdf** that contains all aligned Sources in the order in which the input file is read
- the file **matrix-sensaas.txt** that contains gfit+hfit scores (rows=Targets and columns=Sources)

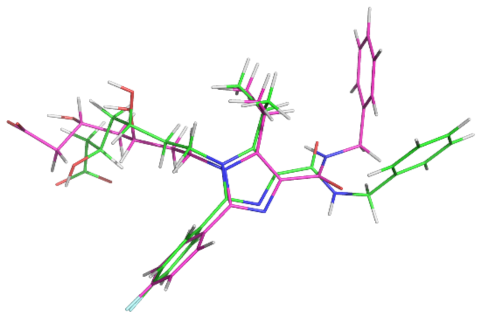
The last line of the output at your prompt may look like:

```
"gfithfit= 0.659 gfit= 0.389 hfit= 0.270 (target 1 ; source 5 ; outputs: bestsensaas.sdf, catsensaas.sdf and matrix-sensaas.txt)
```

It indicates that the best Source solution is the molecule number 5 of the file P04035-7-confs.sdf. This best aligned conformer has a gfithfit score = 0.66.

Visualization You can use any molecular viewer. For instance, you can use PyMOL if installed.

```
pymol P04035-7.sdf bestsensaas.sdf catsensaas.sdf
```



The file bestensaas.sdf (in magenta) is aligned on the experimental conformation P04035-7.sdf (in green).

RMSD calculation

```
rdkit-CalcLigRMSD.py examples/P04035-7.sdf bestsensaas.sdf
RMSD= 2.42
```

Option -l

When executing meta-sensaas.py, you can iterate the alignment by using the option -l:

```
meta-sensaas.py molecules-target.sdf molecules-source.sdf -l 2
```

Here, the calculation is performed twice and only the highest-scoring alignment is selected.

Example:

```
meta-sensaas.py examples/P04035-7.sdf examples/P04035-7-confs.sdf -l 2
```

Post-processing To ease the analysis of the results, the script **ordered-catsensaas.py** can be used to generate files in descending order of score.

```
ordered-catsensaas.py matrix-sensaas.txt catsensaas.sdf
```

or if you want to only retrieve solutions having a gfit+hfit score above a defined cutoff; for example, to select only solutions having a gfit+hfit score >= 0.6:

```
ordered-catsensaas.py matrix-sensaas.txt catsensaas.sdf 0.6
```

- the file **ordered-catsensaas.sdf** contains all aligned Sources in descending order of score
- the file **ordered-scores.txt** contains the original number of Source with gfit+hfit scores in descending order

Visualization You can use any molecular viewer. For instance, you can use PyMOL if installed.

```
pymol P04035-7.sdf ordered-catsensaas.sdf
```

Option -s

When executing meta-sensaas.py, you can also select the score type by using the option -s (default is the score of the Source (**-s source**)):

```
meta-sensaas.py molecules-target.sdf molecules-source.sdf -s mean
```

here the mean of the score of the target and of the aligned source will be used to rank solutions and to fill matrix-sensaas.txt. The option '-s mean' is interesting to favor source molecules that have the same size of the Target. More about Options

Example

```
meta-sensaas.py examples/P04035-7.sdf examples/P04035-7-confs.sdf -s mean
```

b) Clustering mode in order to find alternative alignments

This option allows to repeat alignments in order to find alternative superimpositions when they exist as for example when aligning a fragment on a large molecule. Here, clusters are created based on their distance. It works with one Target and one Source only (the first molecule of each input sdf file is read).

Example (for 20 calculations)

```
meta-sensaas.py examples/P04035-7.sdf examples/phenyl.sdf -r 20
```

You may have to run the script as follows:

```
python meta-sensaas.py examples/P04035-7.sdf examples/phenyl.sdf -r 20
```

Note:
Don't worry if you get the following warning from Open3D: "*Open3D WARNING KDTreeFlann::SetRawData Failed due to no data.*". It is observed with conda on windows.

Here 20 alignments of the Source will be generated and clustered.

Outputs are:

- the file **sensaas-1.sdf** with the best ranked alignment - it contains 2 molecules: first is Target and second the aligned Source
- the file **sensaas-2.sdf** (if exists) with the second best ranked alignment - it contains 2 molecules: first is Target and second the aligned Source
- ...
- file **cat-repeats.sdf** that contains all aligned Sources

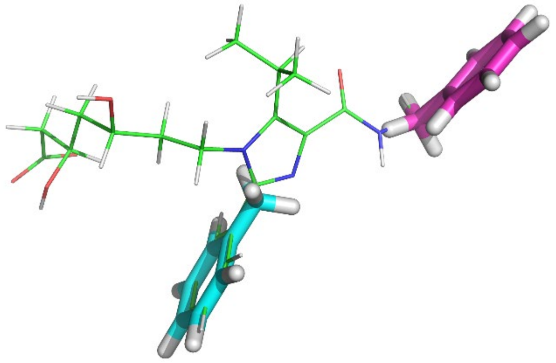
The last line of the output at your prompt may look like:

```
Rank ; Hybrid score gfit + hfit ; Shape gfit ; Color hfit ; percentage of
solutions ; Alignment in SDF file format
1 ; 1.885 ; 0.907 ; 0.977 ; 90.00 ; sensaas-1.sdf
2 ; 1.522 ; 0.692 ; 0.831 ; 10.00 ; sensaas-2.sdf
cat-repeats.sdf contains the 20 sdf solutions
```

It indicates that 2 clusters were created. File sensaas-1.sdf is the best solution of cluster 1 and sensaas-2.sdf is the best solution of cluster 2. Both solutions are perfectly aligned on substructures of P04035-7.sdf.

Visualization You can use any molecular viewer. For instance, you can use PyMOL if installed.

```
pymol P04035-7.sdf sensaas-1.sdf sensaas-2.sdf cat-repeats.sdf
```



The file sensaas-1.sdf (in magenta) and sensaas-2.sdf (in cyan) are aligned on the experimental conformation P04035-7.sdf (in green).

III - Run a flexible alignment with sensaasflex.py

This script only works **with SDF format files.** This script allows to run flexible alignment of two shapes by optimizing the conformer of the Source.

The syntax is:

```
sensaasflex.py <target sdf file (read first structure only)> <source sdf file
(to move; read first structure only)>
```

Example

```
sensaasflex.py examples/P04035-7.sdf examples/P04035-7-confs1.sdf
```

You may have to run the script as follows:

```
python sensaasflex.py examples/P04035-7.sdf examples/P04035-7-confs1.sdf
```

Here, the source file P04035-7-confs1.sdf is aligned (**moved**) on the target file P04035-7.sdf (experimental conformation) (**that does not move**).

- The output file **Source_tran.sdf** contains the aligned (transformed) coordinates of the Source.
- The output file **tran.txt** contains the transformation matrix applied to the input Source file.
- The **slogflex** file details results with final scores of the aligned Source molecule on the last line. The content of the slogflex file may look like (of note, the result may vary from one run to another):

```
“Initial gfit= 0.338 cfit= 0.289 hfit= 0.252 gfit+hfit= 0.59
Round 1 gfit= 0.38 cfit= 0.307 hfit= 0.196 gfit+hfit= 0.576
Round 2 gfit= 0.695 cfit= 0.649 hfit= 0.557 gfit+hfit= 1.252
Best solution (Source_tran.sdf):
gfit= 0.695 cfit= 0.649 hfit= 0.557 gfit+hfit= 1.252”
```

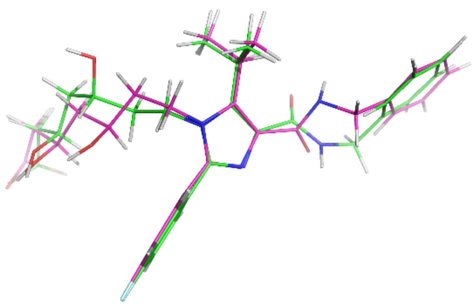
Here, it indicates that the initial rigid alignment gives a solution with a gfit+hfit score = 0.59 and that the alignment obtained after the flexible optimization has a gfit+hfit score = 1.25 which is a much better fitness score.

RMSD calculation (the two molecules are the same; they possess the same 3D graph)

```
rdkit-CalcLigRMSD.py examples/P04035-7.sdf Source_tran.sdf
RMSD= 1.04
```

Visualization You can use any molecular viewer. For instance, you can use PyMOL if installed.

```
pymol P04035-7.sdf Source_tran.sdf
```



The Source_tran.sdf (in magenta) is aligned on the experimental conformation P04035-7.sdf (in green). The RMSD value between conformers equals 1.04Å.

Here we show that SENSAAS-Flex was able to explore the conformational space to find a very close solution to the experimental one. It improves the alignment over the rigid one. Indeed, the first rigid alignment using **sensaas.py** shown above had a gfit+hfit score of 0.608 and a RMSD value of 3.36 Å.

IV - Run flexible alignment(s) with meta-sensaasflex.py

As for meta-sensaas.py, this script only works with **SDF format files**. It allows to align several Source and Target molecules in a **virtual screening** mode or in a **repeat and cluster** mode.

In a virtual screening mode, the syntax is:

```
meta-sensaasflex.py  molecules-target.sdf  molecules-source.sdf  -r  nb  -s
score_type -l x
```

Example:

```
meta-sensaasflex.py examples/P04035-7.sdf examples/P04035-7-confs.sdf
```

This script is suited for performing virtual screenings of SDF files containing several molecules (database mode). For example, if you want to process a sdf file containing several conformers for Target and/or Source.

Please read the above description of **meta-sensaas.py** for more information about options -s or -l, outputs and post-processings (paragraph "**II - Run rigid alignment(s) with meta-sensaas.py**").

In a repeat and cluster mode, the syntax is (for 10 calculations):

```
meta-sensaasflex.py molecules-target.sdf molecules-source.sdf -r 10
```

Example:

```
meta-sensaasflex.py examples/P04035-7.sdf examples/phenyl.sdf -r 10
```

The option -r allows to repeat in order to find alternative alignments when they exist as for example when aligning a fragment on a large molecule.

Please read the above description of **meta-sensaas.py** for more information about option -r, outputs and post-processings (paragraph "**II - Run rigid alignment(s) with meta-sensaas.py**").

Miscellaneous Tools

If you want that **sensaas.py** outputs Target and Source files in pcd and xyzrgb format, set the variable 'verbose' to 1 in the Python script **sensaas.py**. Those format files can be read and visualized using Open3D.

For example:

```
utils/visualize.py examples/VALSARTAN.xyzrgb
```

or:

```
utils/visualize.py examples/VALSARTAN.pcd
```

You can also convert a xyzrgb file into a pdb file for visualization with PyMOL

```
utils/xyzrgb2dotspdb.py examples/VALSARTAN.xyzrgb
```

It will generate the file 'dots.pdb' that can be read with the molecular viewer PyMOL:

```
pymol dots.pdb
```

Colors are class colors as defined above. In our implementation, labels aim to recapitulate typical pharmacophore features such as aromatic (colored in green), lipophilic (colored in white/grey) and polar groups (colored in red).

SENSAAS-Bioisostere

A computational method for 3D shape-guided bioisosteric replacements and scaffold-hopping

Main features:

- Allows bioisosteric replacements and scaffold-hopping

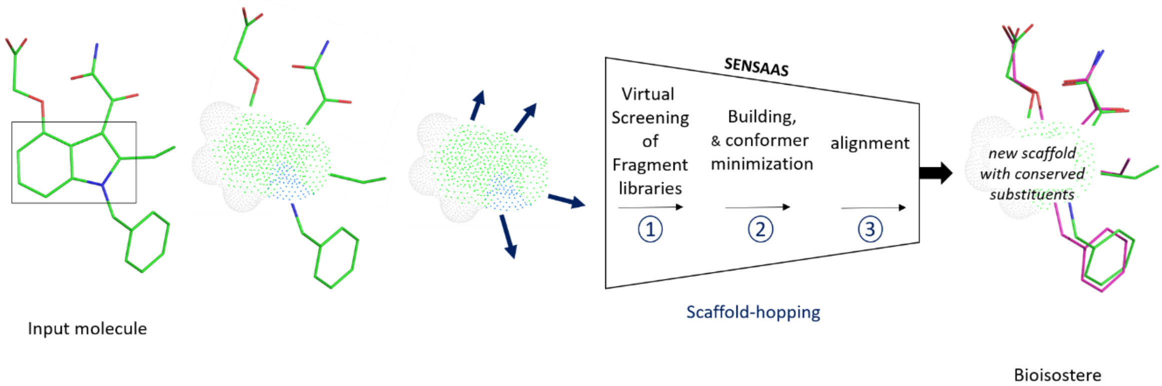
- Bioisosteres are selected based on their 3D shape and pharmacophoric features similarities

- Allows scaffold-hopping with fragments that have more than two substituents

- Provides the superimposition of bioisosteres on the input molecular structure, which is kept as the reference coordinates

- Provides similarity scores

- Is free and open source



This tutorial presents several basic usages of SENSAAS-Bioisostere with SDF molecular files. In the folder examples, you will find some molecular files as test cases to work with.

Example: Scaffold hopping of type 4 to optimize inhibitors of ATR kinase

https://doi.org/10.1021/acs.jmedchem.4c00734

2024

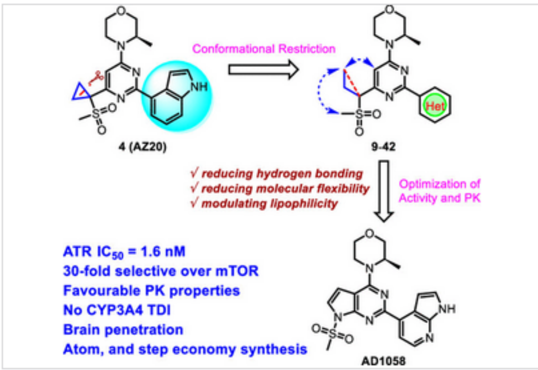
ARTICLE | July 25, 2024

Discovery of Preclinical Candidate AD1058 as a Highly Potent, Selective, and Brain-Penetrant ATR Inhibitor for the Treatment of Advanced Malignancies

Zhi Liu, Kailong Jiang, Yan Liu, Junfei Li, Siqi Huang, Ping Li, Lei Xu, Xiaomin Xu, Xiaobei Hu, Xia Zeng, Zehui Huang, Yubo Zhou*, Jia Li*, Kai Long*, and Mingliang Wang*

Abstract

The ataxia telangiectasia-mutated and Rad3-related protein (ATR) plays a crucial role in regulating the cellular DNA-damage response (DDR), making it a promising target for antitumor drug development through synthetic lethality. In this study, we present the discovery and detailed characterization of AD1058, a highly potent and selective ATR inhibitor, with good preclinical pharmacokinetic profiles. AD1058 exhibits superior efficacy in inhibiting cell proliferation, disrupting the cell cycle, and inducing apoptosis compared to AZD6738. AD1058 displays potent antitumor effects as a single agent or in combination with clinically approved tumor therapies such as PARP inhibitors, ionizing radiotherapy, or chemotherapy in vivo. Considering its enhanced ability to permeate the blood–brain barrier, AD1058 is a promising clinical candidate for the treatment of brain metastases and leptomeningeal metastases in solid tumors. Additionally, among reported ATR inhibitors, AD1058 features the shortest synthesis route and the highest efficiency to date.



Following files are in the folder examples:

The file 7jjg_B_VCD.sdf is the AZ20 molecule extracted from the publication. Coordinates of the ligand (VCD) were extracted from the co-structure PDB 7JJG (https://www.rcsb.org/structure/7JJG). It is the reference (or input) molecule.

The file list-atoms-7jjg_B_VCD contains the number of the atoms of the fragment to replace in molecule 7jjg_B_VCD.sdf.

The file AD1058.sdf is the synthesized bioisostere extracted from the publication and aligned on the AZ20 using SENSAAS-Flex.

1) Create an SDF file containing the fragments that constitute your molecule

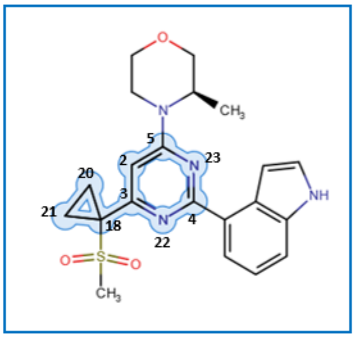
1a- Firstly, use a molecular viewer to identify the number of atoms in the fragment that you want to replace. Then, write each number in a text file, separating them by a space.

In our example, the file list-atoms-7jjg_B_VCD contains the following numbers:

2 3 4 5 22 23 18 20 21

Those numbers correspond to the highlighted atoms in the SDF file 7jjg_B_VCD.sdf as follows:

7jjg_B_VCD.sdf



list-atoms-7jjg_B_VCD

2 3 4 5 22 23 18 20 21

1b- Secondly, use the following script to split the reference molecule into fragments. It tries to identify the fragment to replace by using the list of atoms from the file list-atoms-7jjg_B_VCD.

The syntax is:

```
        perl lea3d-MAKE_FGTS_LIB.pl <reference.sdf> <file containing the list of atoms you want to replace>
```

Example:

```
      perl  lea3d-MAKE_FGTS_LIB.pl  examples/7jjg_B_VCD.sdf  examples/list-atoms-7jjg_B_VCD
```

Here, the script creates 2 files : make_fgts_aggreg.sdf that contains 5 fragments and lib1.sdf that contains 4 fragments.

Each SDF file contains the ensemble of fragments that can be combined to rebuild the reference molecule. This enables one fragment to be replaced by a bioisosteric fragment, after which the molecule can be rebuilt with all the other fragments remaining intact.

The output at your prompt may look like:

lib1.sdf contains 4 fgts

list of atoms to match: 2 3 4 5 22 23 18 20 21 in ../sensaas-bioisostere-main/examples/7jjg_B_VCD.sdf (fragmented in make_fgts_aggreg.sdf)

2 fragments match the list of atoms

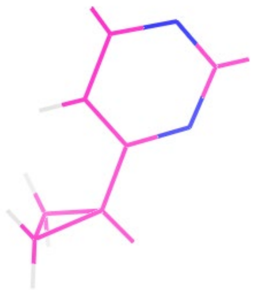
Results:

lib1.sdf fgt 2

make_fgts_aggreg.sdf

The first line after the "Results:" tag indicates that the fragment number 2 in lib1.sdf seems to correspond to the list of selected atoms in text file list-atoms-7jjg_B_VCD.

Visualize the file lib1.sdf by using a molecular viewer and check that the fragment number 2 is indeed the fragment you want to replace. For example, with PyMOL:



```
pymol lib1.sdf
```

Of note, you can choose another fragment to replace by selecting the lib name (lib1.sdf or make_fgts_aggreg.sdf) and by selecting the number of the fragment in the sdf file.

2) Virtual screening of fragment libraries to identify bioisosteres for your selected fragment, followed by alignment of the bioisosteres on the input molecule

The syntax is:

```
perl sensaas-bioisostere.pl <reference.sdf> <file with fragments extracted from reference.sdf> <number of the fragment to replace> <library of fragment to screen>
```

In the db folder, 7 libraries of fragments are available. They were extracted from approved drugs or co-crystallized ligands from the PDBbind dataset. Combinations are substructures composed of 2 or 3 fragments. A test libary called test.sdf is also available in the folder db.

Table 1. Description of the 7 libraries of fragments extracted from bioactive molecules.

Library Name	Number of fragments	Features of the fragments	Source
test.sdf	9		
drug.sdf	4509	single fragments	e-Drug3D (2118 structures)
drug2.sdf	2542	2 combined fragments with MW ≤ 200	
drug3.sdf	2458	3 combined fragments with MW ≤ 200	
pdb.sdf	7990	single fragments with MW ≤ 200	PDBbind (12046 structures)
pdb2.sdf	8678	2 combined fragments with MW ≤ 200	
pdb3.sdf	10333	3 combined fragments with MW ≤ 200	

Example:

In our study, we will use a small library of 9 fragments called test (db/test.sdf):

```
perl sensaas-bioisostere.pl examples/7jjg_B_VCD.sdf lib1.sdf 2 db/test.sdf
```

Note 1: it takes about 1 hour to screen 500 fragments.

Note 2: it has been observed that the minimization with the standard process (RDKit, MMFF94 force field, 2000 Minimization cycles) is sometimes insufficient, resulting in a slight distortion of the structure. Finding a suitable trade-off was necessary here because this step is time-consuming.

The output at your prompt may look like:

perl scripts in ../sensaas-bioisostere-main

SENSAAS-Bioisostere:

Target ../sensaas-bioisostere-main/examples/7jjg_B_VCD.sdf

4 fragments in lib1.sdf - referencex.sdf = libfgt_2.sdf contains the fragment to replace with 3 substituants

Library: 6 fragments in ../sensaas-bioisostere-main/db/test.sdf having >= 3 substitution positions

Screening using metasensaas (option -s mean -l 2)

4 built bioisosteres

Visualize the 4 aligned bioisosteres in ordered-bioisosteres-lib.sdf with PyMol (scores in ordered-bioisosteres-lib-score.txt) - Selected fragments from lib are in ordered-filtered-fragments-lib.sdf (scores in ordered-filtered-fragments-lib-score.txt)

The script creates 2 files: ordered-bioisosteres-lib.sdf that contains best aligned bioisosteres and ordered-bioisosteres-lib-score.txt that contains scores by decreasing order of similarity score.

3) Visualization You can use any molecular viewer. For instance, you can use PyMOL if installed.

```
pymol 7jjg_B_VCD.sdf AD1058.sdf ordered-bioisosteres-lib.sdf
```

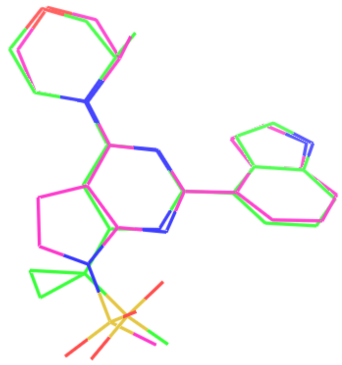


Figure: one bioisostere possess a pyrrolo pyrimidine scaffold (in magenta). It is aligned on the input structure 7jjg_B_VCD.sdf (in green). The fitness score gfit+hfit = 1.54. It has the same molecular structure as AD1058.sdf.

Here, we show that SENSAAS-bioisostere found a pyrrolo pyrimidine fragment to replace the pyrimidine + cyclopropane fragment in AZ20 (7jjg_B_VCD.sdf) thus, reproducing the scaffold-hopping step described in the publication.

When screened against larger libraries (see libraries in folder db), several other fragment bioisosteres can be identified. Those could also be interesting alternatives to the pyrimidine core.